

Adora — Forward-Only Synthesis, Tiling & Memory

As-built notes · Ivan Milic · 2026-09-01 · session summary

This session shipped the forward-only synth, reworked the parallel driver to stream progress, ran the first real MU-RaM cube, and pinned down where the (large) memory goes. Complements the proposal in [adora_5_forward_only.pdf](#) and the CPU scaling in [adora_4_benchmark.pdf](#).

1. Forward-only synth (synth_cube_fwd) — as-built

Added to [synth3d.py](#): factored the vmap body into `_synth_cube`; `synth_cube` is unchanged (differentiable-ready), and the new twin is

```
synth_cube_fwd = jax.jit(lambda ...: jax.lax.stop_gradient(_synth_cube(...))).
```

`stop_gradient` makes it a *constant* to any outer torch/jax autodiff — no tape is ever built. **Verified:** bit-identical to `synth_cube` (max abs diff 0.0), and `jax.grad` through it is exactly 0 while `synth_cube` still differentiates. **Deviation from the proposal:** the `dtype=float32` knob was *dropped*. With `x64` on and `float64` constants baked into the shared kernel (`jnp.eye(4)` in the DELO solver, Planck consts in [lineop.py](#)), `f32` inputs promote straight back to `f64` — it rounded inputs ($\sim 0.3\%$) with *no* memory win. A real `f32` path needs a `dtype-generic` kernel (deferred; out of scope for “don’t touch the physics”).

2. Parallel driver (synth_cube_mp) — reworked

[synth3d_mp.py](#) now uses a `spawn Pool` with an `initializer`: each worker pins its core(s), imports JAX, and reads the line list *once*, storing them in a per-process dict and reusing them across many **tiles**. Columns are split into fixed-size tiles (~ 4 /worker by default), streamed back with `imap_unordered` and placed by start index, and wrapped in a `tqdm` bar so progress is visible during the run (silent fallback if `tqdm` is absent). Each worker runs `synth_cube_fwd` (mp output is plain numpy across a process boundary $\Rightarrow$ forward-only). New kwargs: `tile` (columns/task) and `progress`. Output bit-identical to single-device; 40×40 on 32 workers ≈ 35 s (spawn+compile+run), a touch faster than before (one line-list read/worker, warm JIT reuse). Guard footgun unchanged: callers **MUST** be under `if __name__=="__main__"`.

3. Where the memory goes (the “huge” peak)

Inside a worker, one tile is a *single* triply-nested `vmap` — over **columns** (the tile), then **wavelength**, then **depth** — fused into one XLA call. So the opacity/emissivity intermediates (`eta`, `chi`, Voigt/`exp` temporaries) are all live at once for the whole tile $\times n_\lambda \times n_z$ batch. Hence

$$\text{peak RSS/worker} \approx \underbrace{\sim 1 \text{ GB}}_{\text{JAX baseline}} + K \cdot \text{tile} \cdot n_\lambda \cdot n_z, \quad \text{aggregate} = n_{\text{workers}} \times (\dots).$$

Measured on the first real cube ([3D_full_subdomain](#), iter 0, `muramsub`, `skip=4` $\Rightarrow \sim 141$ k columns, $n_\lambda=201$): the auto default `tile` ≈ 1100 gave **~ 17 GB/worker**, and with 32 workers loosely in sync the tree **peaked at ~ 540 GB** of the 1006 GB node — spiky (dropping to ~ 170 GB between tiles as intermediates free). The run finished fine and saved a **905 MB** Stokes cube; memory released cleanly.

tile is the memory lever (peak is linear in it; total work is unchanged — tiles just stream through the warm workers).

tile	$\sim$ peak/worker	note
1100 (auto)	~ 17 GB	measured; $\times 32 \rightarrow \sim 540$ GB
128	~ 3 GB	projected (linear)
16 (current)	~ 1.2 GB	<code>nworkers=128</code> : ~ 150 GB

Caveat. The old default (`tile = n_{px}/(4n_{\text{workers}})`) was tuned for bar granularity, not memory; it scales badly for big maps and would **OOM the node** at `skip=1`. n_λ and n_z are fixed by the physics — `tile` is the only cheap dial.

4. How to run & what’s next

- **Current CLI** ([muram_loader.py](#)): `load_muram_cube(..., skip=2)` then `synth_cube_mp(waves, dz, cube, nworkers=128, tile=16)` $\rightarrow$ saves `/dat/milic/synthesized_cube.npy`. Small `tile` keeps per-worker RSS ~ 1 GB; pick `tile` from a memory budget, not pixel count.
- **Recommended:** make the *default* `tile` memory-bounded (cap from an $n_\lambda n_z$ budget targeting ~ 1 – 2 GB/worker) so an unattended full-res run can’t blow up. Not yet done.
- **Still open:** validate the loader on real data (orientation/velocity/field signs vs Lightweaver); a **persistent pool** to kill the per-call cold-start for inversions; inclined rays; response cube.